Firmware & Service Supplement

Image Construction, Calibration Security, Memory Reset and Undocumented Commands

Reverse-Engineering Supplement — not an Agilent/HP publication
Document 531XX-90RE1
Compiled by Jared Cabot • jetstreamtechnology@protonmail.com
Firmware reverse-engineering and analysis by Claude (Anthropic)
Derived from firmware disassembly and the HP/Agilent manuals
Revision 1.0 • August 2026

Scope

This supplement documents the internal firmware of the Agilent/HP 53131A, 53132A and 53181A counters as it bears on three service tasks: restoring a serial number said to be lost after a firmware update, recovering a forgotten calibration security code, and using interface and front-panel features not covered by the standard manuals — including how the optional Channel 3 prescaler board is identified and configured. It supplements, and does not replace, the Operating Guide (53131-90055 for the 53131A/53132A, 53181-90001 for the 53181A), the Programming Guide (9018-01066) and the Assembly-Level Service Guide (9018-01065). Routine front-panel operation — the Calibration menu and the Utility (self-test) menu reached by holding a key at power-on — is documented in the Operating Guide, and is described here only where the firmware behaviour extends beyond it.

Basis

The hardware registers, memory addresses, key codes and command keywords in this document describe the counter firmware carried in four 128 KB EPROMs (U8–U11) per instrument, across firmware revisions 3427 through 4613. The primary reference image is the 53132A revision 4613; where a value differs by model or revision it is called out.

Applicability

This supplement covers the 53131A (rev 3427, 3646, 3944, 4243), the 53132A (rev 3703, 3944, 4613) and the 53181A (rev 3703, 3944, 4243). Boot behaviour, the keyboard matrix decode and the memory-clear mechanism are identical in structure across all three models; where a value differs by model it is called out.

Conventions

Hexadecimal values carry a $ prefix ($172C); all addresses are byte addresses in the CPU's 24-bit space after firmware is mapped. Commands and machine-readable text are shown in a monospaced face. References to the manufacturer's manuals use the short forms “Operating Guide”, “Programming Guide” and “Service Guide”. “NVRAM” and “EEPROM” are used interchangeably for the instrument's non-volatile store, as the firmware itself does.

Disclaimer

This is an independent reverse-engineering document. It is not a product of Agilent Technologies, Hewlett-Packard or Keysight, carries no endorsement, and may contain errors. The procedures in Chapters 3 and 5 can erase your instrument's calibration and configuration. A full memory clear is irreversible and will require the instrument to be recalibrated against traceable standards. Proceed only if you understand and accept that consequence.

Contents

1

General Information

Instrument and Firmware Family

The Agilent/HP 53131A and 53132A are 225 MHz (standard) universal counters; the 53181A is a frequency counter of the same generation and mechanical design. All three share one controller board, one firmware architecture, and one manual set. The firmware for each is carried in four 1-Mbit EPROMs (AM27C010 or M27C1001, PLCC-32) designated `U8`, `U9`, `U10` and `U11`, each holding 128 KB.

Several firmware revisions of each model exist. The revision number is embedded in the firmware itself and is returned as the last field of the `*IDN?` response.

Table 1-1. Firmware Revisions Covered

Model	Revisions present	Branding	EPROM part
53131A	3427, 3646, 3944, 4243	HP / Agilent	53131-800xx
53132A	3703, 3944, 4613	HP / Agilent	53132-800xx
53181A	3703, 3944, 4243	HP / Agilent	53181-800xx

Revisions 3317, 3335 and 3402 (noted in the Service Guide as the earliest builds) predate the calibration-security feature; on those units the `CAL:` item lives in the Utility menu and no code is required. Everything in Chapter 3 applies to rev 3427 and later.

Processor and Memory Map

The controller is built around a **Motorola MC68331** — the parts list specifies the `MC68331CEH16` — a CPU32 core (a 68020-class instruction subset) with an on-chip System Integration Module and a General-Purpose Timer. The reset code is consistent with this part: it programs the 68331's SIM registers — the System Protection Control Register at `$FFFA21`, and the chip-select base/option registers from `$FFFA44` onward, including the boot chip-select `CSBARBT` at `$FFFA48`. The self-test strings further name the on-chip QSPI and an external FPGA.

Reset vectors sit at the base of the ROM: the initial stack pointer is `$0010FFFE` and the initial program counter is `$0000172C`. From these and the register programming the map below is derived. It is the same for all three models.

Table 1-2. Controller Memory and I/O Map

Range	Contents
$000000–$07FFFF	Program ROM, 512 KB (the four EPROMs)
$100000–$10FFFF+	Static RAM; system globals based at $100000
$300000–$300FFF	Front-panel keyboard matrix ports (6 bits each)
$400000–$401FFF	Non-volatile store: AT28C64B-class 8Kx8 parallel EEPROM (chip select 9)
$FFF900–$FFF9FF	MC68331 QSM port pins (part of the board-present sense)
$FFFA00–$FFFA7F	MC68331 SIM registers (SYPCR, chip selects)
$FFFC00–$FFFC1F	MC68331 QSM / QSPI (serial link to the trigger-level DACs)

Note

The non-volatile calibration and configuration store is a **parallel EEPROM of 8 KB mapped at $400000** (68331 chip select 9), not a serial device on the QSPI bus — the QSPI carries only the trigger-level DACs. The EEPROM holds a validity signature (the ASCII string `HP53131 is a winner!` at $400002) and a checksum at $400000, followed by the calibration factors, the security code, the saved configuration and the Channel 3 option setting. The firmware caches it in RAM and writes changed bytes back to the device.

The part is an **Atmel AT28C64B** (or a pin-compatible 8K x 8 parallel EEPROM); the 8 KB chip-select window and the write routines match it exactly. Reads are ordinary memory accesses. Writes go through two small ROM routines: a single-byte write at `$000400` and a page write at `$00043C`, both of which start the internal write cycle and then confirm it by **DATA polling** — reading the just-written cell back until it returns the written value, with a bounded timeout and up to five retries. Page writes are held within a 16-byte boundary. The firmware issues **no software-data-protection unlock sequence** before writing, so the device is used unprotected: it can be read, dumped and reprogrammed with an ordinary EEPROM programmer, and writes from the pForth monitor (Chapter 8) take effect immediately.

Non-Volatile Memory Layout

The 8 KB EEPROM is organised as a checksummed 32-byte header followed by fixed-size calibration/configuration records. The header is the only part with a simple byte layout; the records are bit-packed.

Table 1-3. EEPROM Header Record ($400000-$40001F)

Offset	Size	Contents
$00	2	Header checksum: 16-bit sum of bytes $02-$1F (big-endian)
$02	20	Validity signature, ASCII "HP53131 is a winner!"
$16	4	Channel 3 32-bit configuration value
$1A	1	Channel 3 status / revision byte
$1B	1	Channel 3 option code (bits 0-6) and coupling (bit 7)
$1C	1	Channel 3 prescaler ratio divided by 128

From `$400020` onward the calibration factors, the security code, the saved/recalled setups and the configuration are held in 64-byte records. The store keeps two record slots, at `$400020` and `$400060`, and writes them alternately: each new copy is written to the other slot and stamped with a higher generation number than the one it supersedes, so an interrupted or failed write cannot destroy the last good copy. Every slot begins with the fixed 12-byte header below.

Table 1-4. Non-Volatile Record Header (per 64-byte slot)

Offset	Size	Contents
+$00	2	Tag word. $5313 marks a valid record; a superseded or cleared slot has this tag invalidated to $FF13 (high byte set to $FF).
+$02	2	Record checksum: 16-bit sum of bytes +$08 through +$3F.
+$04	4	Generation counter. The valid slot holding the highest value is the active record.
+$08	4	ASCII firmware revision that wrote the record (for example "3944", "4243").
+$0C	52	Bit-packed calibration and configuration variables.

The revision field makes each slot a record of the firmware that last wrote it, so a unit that has been through a firmware update carries the two revisions in its two slots. Within the packed area the individual variables are **bit-packed** to widths taken from a descriptor table, and each has an associated minimum and maximum limit. A consequence worth noting for repair work: values such as the calibration security code (held at RAM `$100184`) do not sit at a fixed byte offset in the dump and cannot be located or edited by hand — they are packed bitfields, validated by the record checksum and the limit tables.

Note

The memory clear of Chapter 3 acts on exactly this structure: the routine at `$02D736` invalidates the `$5313` tag of both slots, which leaves no valid record and forces the firmware to rebuild the store from defaults at the next power-up.

Note

This is why the calibration data and the security code can only be reset together, by clearing the whole store (Chapter 3), and why the accompanying EEPROM tool edits only the plain header fields — the Channel 3 option — and not the packed calibration records.

2

Firmware Image Construction

EPROM Set and Bus Organization

To read the firmware as code, the four EPROM images must be combined in the correct order. The MC68331 has a 16-bit data bus, and the counter drives it from **two banks of two devices** — not, as the four-device count might suggest, a single 32-bit-wide array. Each bank supplies one 16-bit word: an even device drives D15–D8 and an odd device drives D7–D0.

The mapping is:

Table 2-1. EPROM to Address-Space Mapping

Device	Bank	Byte lane	Combined address range
U8	Low	D15–D8 (even)	$000000–$03FFFF
U10	Low	D7–D0 (odd)	$000000–$03FFFF
U9	High	D15–D8 (even)	$040000–$07FFFF
U11	High	D7–D0 (odd)	$040000–$07FFFF

In words: U8 and U10 interleave to form the lower 256 KB, U9 and U11 the upper 256 KB. Any other pairing yields either unreadable text or an implausible reset vector.

Reconstructing a Linear Image

The following procedure produces one 512 KB big-endian image with ROM at offset zero, suitable for a CPU32/68020 disassembler.

1. For the low bank, interleave U8 (even) and U10 (odd) byte by byte: output[2i] = U8[i], output[2i+1] = U10[i], for i = 0 to 131071. This yields 256 KB.
2. For the high bank, interleave U9 (even) and U11 (odd) the same way. This yields a second 256 KB.
3. Concatenate: low bank first ($000000), high bank second ($040000).
4. Disassemble as Motorola CPU32 (68020 instruction set), big-endian, load address $000000.

```
# Python, per instrument -- Ux are the four 131072-byte EPROM images
def bank(even, odd):
    out = bytearray(2*len(even))
    for i in range(len(even)):
        out[2*i]   = even[i]     # D15-D8
        out[2*i+1] = odd[i]      # D7-D0
    return bytes(out)

image = bank(U8, U10) + bank(U9, U11)   # 524288 bytes, ROM at $000000
```

Note

Some 53131A revision-4243 images are distributed as Intel-HEX text rather than raw binary; convert each to a 128 KB binary before interleaving. The byte order and bank assignment are unchanged.

Verifying the Reconstruction

A correct image satisfies four checks:

- The long-word at $000000 is the initial stack pointer, `$0010FFFE`; the long-word at $000004 is the reset PC, `$0000172C`.
- Disassembly at `$172C` begins with a run of `move` instructions writing the 68331 SIM chip-select registers at `$FFFA44` upward — a textbook CPU32 boot.
- The copyright string is intact and readable near `$000E11` (“Copyright Agilent Technologies 1993” on later revisions, “Hewlett-Packard Co. 1993” on earlier ones).
- The four-digit firmware revision appears as ASCII near `$000E0A` and again inside the `*IDN?` builder.

Caution

Programming EPROMs with the two banks swapped, or with an even/odd device transposed, produces a unit that will not boot. Confirm the reset vector reads `$0010FFFE / $0000172C` in your combined image before burning devices.

3

Calibration Data and Security

The Security Code

From revision 3427 onward the counter protects calibration with a numeric security code. While the instrument is *secured*, calibration routines and the writable calibration data are refused; *unsecuring* requires the code. The factory-default code is the model number:

Table 3-1. Factory-Default Security Codes

Model	Default code
53131A	53131
53132A	53132
53181A	53181

The code and the secured/unsecured state live in the $400000 non-volatile EEPROM and survive power-off and remote reset. Ordinary use of the security system — securing and unsecuring with a code you know, changing the code and reading the calibration count — is the documented Calibration menu (hold SCALE/OFFSET at power-on), covered in the Operating Guide under “Using the Calibration Menu.” This chapter deals with what that menu does not: recovering a *forgotten* code, and the layout of the calibration data itself. There are three ways to get past a forgotten code, in increasing order of destructiveness.

Recovery by Remote Command

If the code is merely mislaid but calibration data is intact, and you can find the code, the cleanest route is the documented remote interface. These commands ARE in the Programming Guide and are listed here for completeness:

```
:CALibration:SECurity:STATe?
:CALibration:SECurity:STATe OFF, <present_code>
:CALibration:SECurity:STATe ON, <present_code>
:CALibration:SECurity:CODE <new_code>
```

Query `:CAL:SEC:STAT?` returns 1 when secured. Sending `:CAL:SEC:STAT OFF, <code>` with the correct present code unsecures the unit; while unsecured, `:CAL:SEC:CODE <new_code>` installs a new code. None of this helps if the present code is genuinely unknown — for that, read on.

Recovery by Reading the Code

The firmware contains a hidden service function that **displays the current security code in clear**, without changing anything. It is reached from the hidden debug menu described in Chapter 5 (hold the `+/-` key while switching power on), whose `CAL CODE ?` item prints `CAL CODE IS <n>`. This is the least destructive recovery: it preserves calibration, configuration and the calibration count.

The code itself is held at RAM `$100184` and mirrored, with the rest of the calibration data, in the bit-packed non-volatile records at `$400020` onward (Chapter 1). Because those records are bit-packed rather than stored at a fixed byte, the code cannot be read straight out of an EEPROM dump — reading it on the instrument, through this menu, is the practical route.

Note

The debug menu is gated (Chapter 5); if it does not open, enable it first with the remote command `:DIAGnostic:SYSTem:HMACCESS`.

Recovery by Memory Clear

If the code cannot be read or the calibration store is already corrupt, the firmware provides a full non-volatile-memory clear that resets the security code to the model default of Table 3-1. This is the "confidential procedure" the Service Guide alludes to but does not print. It is triggered entirely from the front panel at power-on:

Procedure — Clear Non-Volatile Memory

1. With the instrument switched off, press and hold the LEFT ARROW and RIGHT ARROW keys together.
2. Switch the instrument on while continuing to hold both keys.
3. The display shows EEPROM CLEAR. Release the keys.
4. Allow the instrument to complete its start-up. The security code is now the model default (53131 / 53132 / 53181).

The underlying mechanism is as follows. At power-on the keyboard is scanned; the simultaneous LEFT+RIGHT arrow press produces internal key code `$1F` on the 53131A/53132A (`$1A` on the 53181A, whose matrix is smaller). The key-decode at `$038FF4` sets the clear-request flag at `$100822`. Early in start-up the boot code at `$000DEC` tests that flag and, when it is set, calls the routine at `$02D736`, which **invalidates the `$5313` tag** on both non-volatile record slots (at `$400020` and `$400060`, writing `$FF` over the high tag byte). With no valid record left, the firmware rebuilds the store from defaults — including the model-default security code — at the next start-up. Separately, the boot path at `$001044` scrolls the notice `EEPROM CLEAR` across the display and holds it there by calling the display-delay routine at `$001332` with a duration of two seconds (the float `$40000000` = 2.0). Why a two-key combination and not a single key is explained in Chapter 5.

Note

The instrument's display is a single 12-character line, so any message longer than that — `EEPROM CLEAR`, `CAL CODE IS <n>`, the menu banners — is shown by scrolling it across the line. The routine at `$001332` takes a floating-point number of seconds and paces that scroll; it performs no memory operation.

Caution

The memory clear is irreversible. It resets the security code but also clears calibration factors, save/recall registers and stored configuration. The unit will report itself uncalibrated and must be recalibrated against traceable standards. Before clearing, if the unit still communicates, read and save the calibration data with `:CALibration:DATA?` as the Service Guide recommends.

Note

The calibration count (`CAL COUNT?`, also on the debug menu) is NOT reset to zero by the clear; it continues to increment, so a cleared unit does not masquerade as factory-fresh.

The companion tool `531xx_eeprom_tool.py` performs the equivalent of this clear on a saved EEPROM dump — it blanks the image so the counter rebuilds its non-volatile memory with the default code on next power-up — after making a backup first. It cannot reset the code while preserving calibration, for the same reason the on-instrument clear cannot: the two live in one checksummed store.

Calibration Data Structure

The calibration constants live in the non-volatile store from `$400020` onward, in the 64-byte `$5313`-tagged records of Chapter 1. Within a record the values are bit-packed to widths taken from a descriptor table, so they cannot be read straight from a memory dump — but the firmware exposes them in human-readable form through a set of queries:

`:CAL:DATA?`	The portable calibration block — the same data that `:CAL:DATA <block>` restores. On the 53131A it is a 56-byte definite-length block.
`:CAL:COUN?`	The calibration count — it increments on each calibration and is never reset (a cleared unit keeps its count), so it shows how many times a unit has been adjusted.
`:DIAG:CAL:STAT?`	Calibration validity (`1` = valid).
`:DIAG:CAL:TINT:DATA?`	The time-interval calibration constants, as integers.
`:DIAG:CAL:INT:DATA?`	The interpolator calibration constants, as integers.
`:DIAG:CAL:ROSC:DATA?`	The reference-oscillator calibration — returns `-241` (hardware missing) when no oven oscillator is fitted.

The `:CAL:DATA?` block decodes as 14 big-endian 32-bit integers. A read from one calibrated 53131A (calibration count 18) decodes as follows — the exact values are unit-specific:

Table 3-2. :CALibration:DATA Block Fields

Word	Example	Meaning
0, 1	2048, 2048	Channel 1 / 2 offset trim, near 12-bit mid-scale ($800). Word 0 moved when :DIAG-level offset cal was re-run on Channel 1.
2, 3	1744, 1751	Channel 1 / 2 gain constants.
4-11	2162 … 1884	Time-interval calibration constants (= :DIAG:CAL:TINTerval:DATA?).
12	2048	Reference / DAC mid-scale.
13	79047	Trailing validation word (block checksum).

Words 4-11 are the eight time-interval constants that set the counter's single-shot timing resolution; they match `:DIAG:CAL:TINTerval:DATA?` exactly, which is simply the readable form of the same data. The `2048` words are 12-bit DAC references at mid-scale (nominal / 0 V). Restoring a saved block with `:CAL:DATA` writes the set back, so a backup taken with `:CAL:DATA?` before any service work preserves the unit's adjustment.

Caution

A `:CAL:DATA` block encodes one specific instrument's adjustment. Do not load a block captured from a different unit — it will mis-calibrate the target.

4

Model and Serial Number Identity

The *IDN? Response

The instrument reports its identity through the standard `*IDN?` query. The response is built from a fixed template compiled into each firmware image; in every revision of every model it has the form:

`HEWLETT-PACKARD,<model>,0,<revision>`

The manufacturer field, the model and the third field are literal text in the ROM; only the four-digit firmware revision is substituted at run time. The Programming Guide gives the same template — its example shows `HEWLETT-PACKARD, 53131A,0,XXXX`, with the third field fixed at zero.

Table 4-1. *IDN? Template, by Image

Model / rev	Embedded *IDN? template
53131A 3427	HEWLETT-PACKARD,53131A,0,<rev>
53131A 4243	HEWLETT-PACKARD,53131A,0,<rev>
53132A 3944	HEWLETT-PACKARD,53132A,0,<rev>
53132A 4613	HEWLETT-PACKARD,53132A,0,<rev>
53181A 4243	HEWLETT-PACKARD,53181A,0,<rev>

There Is No Electronic Serial Number

The third `*IDN?` field is the serial-number position, and it is the constant character `0` in every image. The firmware contains no “serial” or “S/N” tokens, no SCPI protected-user-data command (`*PUD`), and no code path that reads or writes a serial number. The serial number exists only on the rear-panel label; the instrument holds no electronic copy, and none can be programmed in. A firmware update therefore cannot erase a serial number — there was never one stored to lose. For asset tracking the serial must be kept in an external record keyed to the interface address.

Note

A unit that returns a non-zero serial in `*IDN?` would indicate a firmware variant outside those described here.

Model Identity Is Firmware-Defined

The model name in `*IDN?` is not sensed from the chassis — it is compiled into each firmware image. A set of, for example, 53132A EPROMs installed in a 53131A chassis makes the unit identify as a 53132A. If a unit's reported model changes after an EPROM update, the installed image does not match the chassis; programming the matching model's EPROM set (Chapter 2) restores the correct identity.

5

Service and Diagnostic Menus

Power-On Key Modes

The firmware reads the front panel during start-up and, depending on which key is held, enters one of several special modes. Two are documented in the Operating Guide (the Calibration menu and the Utility / self-test menu); the rest are not. The dispatcher is at `$0372A4`: each key is decoded to an internal key code that sets a flag, and the dispatcher routes to the matching handler. The mapping is:

Table 5-1. Power-On Key Modes (hold during power-on)

Key held	Mode entered	Handler	Gated	Documented
SCALE/OFFSET	Calibration menu	$0301DE	no	yes
RECALL	Utility / self-test menu	$034CF0	no	yes
+/-	Hidden debug menu	$033906	yes	no
TRIG/SENS 1	Service menu	$034098	yes	no
TRIG/SENS 2	Option / interface menu	$03295A	yes	no
LEFT + RIGHT arrows	Clear non-volatile memory	$02D736	no	no

The **Documented** column records whether the mode appears in the standard manuals: the Calibration and Utility menus are described in the Operating Guide; the remaining four are not documented by the manufacturer and are the subject of this chapter.

The three gated modes — the debug, service and option menus — open only when the hidden-menu-access flag (RAM `$10019B`, a volatile session flag cleared to zero at every power-on) is set; until then, holding those keys does nothing. The Calibration menu, the Utility / self-test menu and the memory clear are **not** gated and are always available. The gate is set by the remote command described next.

What each mode opens

- **SCALE/OFFSET — Calibration menu.** The calibration and security menu. Documented — see the Operating Guide, “Using the Calibration Menu.”
- **RECALL — Utility / self-test menu.** Firmware revision, GPIB address, timebase source and the built-in self-test. Documented — see the Operating Guide, “Using the Utility Menu.”
- **+/- — Hidden debug menu.** A gated engineering menu of roughly ten items — non-volatile memory dump, calibration-code readout, model select and the pForth monitor. Its items are detailed under “The Hidden Debug Menu” below (Table 5-2).
- **TRIG/SENS 1 — Service menu.** In these firmware revisions this scrolls the banner `SERVICE MENU` and returns; it carries no selectable items. See “The Service Menu” below.

- **TRIG/SENS 2 — Option / interface menu.** The `NO OPT MENU`, an interactive menu with two items: `OPT:` (view / set the Channel 3 option, Chapter 7) and `GPIB ENABLED` (interface enable state). See “The Option / Interface Menu” below.
- **LEFT + RIGHT arrows — Clear non-volatile memory.** The full NVRAM clear (Chapter 3); the two-key chord makes a key code no single key can produce.

Enabling the Menus Remotely

The gate at `$10019B` is written by an undocumented remote command, `:DIAGnostic:SYSTem:HMACCESS` ("hidden-menu access"). Its handler is the firmware routine `set_vt_hma` at `$037024`, which stores the command's argument directly into the gate byte. Once set, the debug, service and option power-on menus open. **The gate does not persist.** It is a plain RAM byte: disassembly shows the `HMACCESS` command is the *only* code that writes `$10019B`, it is not one of the bit-packed variables the non-volatile record stores, and start-up clears it to zero — so the menus re-lock on every power cycle and `HMACCESS` must be re-sent once per session before the gated menus will open.

Note

The `set_vt_hma` handler does also call `save_nv`, but that routine commits the standard non-volatile variable set (calibration and configuration) — not the gate byte, which it never touches. Verified on the bench: after a power cycle the `+/-` menu is locked again until `HMACCESS` is re-sent, whereas settings that *are* part of the stored record (the `PFORTH:` toggle, for example) do survive. This is a sensible safety property — the hidden menus cannot be left permanently unlocked — not a fault.

This is the intended service path: enable menu access over the bus, then use the front-panel menus. `:DIAGnostic:SYSTem:HMACCESS` is one leaf of a large undocumented `:DIAGnostic` service tree — the subject of Chapter 6. The argument is a small integer written verbatim to the gate byte; `1` enables menu access.

The Keyboard Matrix

The front panel is a matrix read through two 6-bit ports, `$300001` (columns) and `$300000` (rows). Each port's value indexes a decode table at `$3391BA` to a line number 0–6, and the two line numbers index a 7×7 grid at `$3391FA` that yields the key code. On a normal single key-press exactly one line is low in each port. This detail explains the memory-clear combination.

The LEFT and RIGHT arrow keys share a row line but sit on adjacent column lines. Pressing both pulls two column lines low at once, a pattern that no single key can make; the decode table maps it to a phantom seventh column, and the grid cell at that phantom column carries key code `$1F` on the 53131A/53132A. That code exists *only* as the two-arrow chord, which is why it was chosen for the destructive memory clear — it cannot be produced by accident.

Note

On the 53181A, whose front panel has fewer keys, the same two-arrow chord produces code `$1A` and the boot code compares against that value instead. The physical action — hold both arrows at power-on — is identical on all three models.

The Hidden Debug Menu

The debug menu (hold +/- at power-on) is the richest of the gated menus: an interactive list built by the menu engine at $035C32 from the descriptor at $033E74, ten items deep, with the prompt line MENU:. Navigation differs from the ordinary front-panel menus: the +/- **key steps** from one item to the next, and the **arrow keys act on the current item** — on a query item (ending ?) an arrow or Enter shows the value, and on a toggle item (ending :) the arrows switch it ON / OFF. The items, in the order they appear, are:

Table 5-2. Hidden Debug Menu Items (in menu order)

Item	Function
MLB TEAM ?	Engineering easter egg — scrolls the original development team's names (KRISTI BITTNER, LEE COSART, …). Harmless.
MODEL ?	Shows, and can re-select, the stored model identity (Chapter 4). The internal build tag reads SHORTSTOP -- 53131 on the 53131A (BASEHIT -- 53132 on the 53132A).
DEBUG O/P:	Diagnostic-output enable (default OFF).
PFORTH:	Toggle that enables the embedded pForth monitor on the serial port (default OFF; Chapter 8). It does not open on the front panel.
DEBUG OS:	Second diagnostic-output toggle, operating-system trace (default OFF).
GPIB BUF:	GPIB input-buffer view, for protocol debugging (default OFF).
SHOW PLUS:	Whether leading “+” signs are shown on results (default OFF).
DISPLAY:	Front-panel display enable (default ON).
CAL CODE ?	Displays the calibration security code in clear; the result scrolls as CAL CODE IS <n> (Chapter 3). Read-only.
NV COUNT ?	Shows the write counts of the two non-volatile record slots, for example NV0: 0000001EH NV1: 000000E7H — the two 64-byte slots of Table 1-4 (the higher count is the active slot).

Every item label above is taken from a running instrument and matches the firmware's debug-menu string pool at $033EB4 onward; the trailing #! control bytes that mark a scrolling line are not shown.

The Service Menu

Holding TRIG/SENS 1 at power-on enters the handler at $034098. In every firmware revision examined (3427–4613) this handler does only one thing: it passes the string SERVICE MENU to the banner routine at $037136, which scrolls it once, and returns to normal operation. Unlike the debug and option menus it does **not** call the interactive menu engine and presents **no selectable items** — it is an entry point that was reserved but left empty in the shipped firmware. It is listed here for completeness so that a service technician who finds it is not left searching for items that do not exist.

The Option / Interface Menu

Holding TRIG/SENS 2 at power-on enters the handler at $03295A, which builds the NO OPT MENU through the same interactive engine as the debug menu ($035C32, descriptor at $032A66). It has two items:

OPT:	Displays, and sets, the stored Channel 3 option code — the same value *OPT? reports and :DIAGnostic:OPTion:HFR writes (Chapter 7). It is the front-panel route to declaring which prescaler board is fitted, with no GPIB controller needed. Stepping onto the item shows the current option (for example OPT: 030); the arrow keys cycle it through the option codes, and the value is committed to the counter's non-volatile store.
GPIB ENABLED	Shows, and toggles, whether the GPIB (IEEE-488) interface is enabled. With it disabled the counter ignores the bus and operates front-panel only.

Enabling or changing the Channel 3 option

Because the option is stored in the counter rather than on the board (Chapter 7), a board swap needs the stored option updated to match. The OPT: item does this from the front panel:

1. Enable the gated menus first — send :DIAGnostic:SYSTem:HMACCESS 1 over the bus (this section, earlier). Without it the TRIG/SENS 2 menu will not open.
2. Switch the counter off; press and hold TRIG/SENS 2; switch on. The NO OPT MENU appears.
3. Step with the +/- key to OPT:. The display shows the currently stored option.
4. Use the arrow keys to cycle OPT: to the option that matches the fitted board (015, 030, 050, 124, …). The selection is written to non-volatile memory.
5. Switch the counter off and on normally so the boot code re-reads the stored option; confirm with *OPT? or a known Channel 3 signal.

Note

The OPT: item sets the option *code* (the $1B field). The per-board prescaler ratio (the $1C byte, Chapter 7) is a separate stored value; for a board whose ratio differs from the option's usual default, set both together with :DIAGnostic:OPTion:HFR <prescale>,<Nxxx>,<coupling> (Table 7-5), which remains the complete route.

Note

The menu title NO OPT MENU is the firmware's own label; it appears even when an option board is fitted. When the internal “no option” flag at $1038DA is set the handler shows only the scrolling title and skips the two items, which is the origin of the name.

Caution

The debug menu exposes direct non-volatile-memory readout (NV COUNT ?), a live interpreter (PFORTH:) and writable toggles, and the option menu writes the stored option code. Both can change calibration and configuration with no confirmation prompt. Use them read-only unless you intend to change stored data, and read Chapter 3's caution first.

6

Undocumented Commands and Features

The Command Dictionary

The SCPI parser matches against a keyword dictionary held near `$4DCC4` in the revision-4613 image, with each keyword stored as a short form plus its long-form completion. A number of these keywords are understood by the firmware but appear in neither the Programming Guide nor the Operating Guide. Most belong to a service branch of the `:DIAGnostic` subsystem; a few are measurement-function tokens outside the documented list. The command *tree* — which keyword is a child of which — is held in a separate packed table, so the exact command path of a service keyword may differ from the grouping shown.

Caution

The commands in this chapter are service and diagnostic controls. They can drive DACs, force front-end states and start calibration routines directly. Sending them to a working, calibrated instrument can disturb its calibration. The exact argument syntax of the DIAGnostic service commands is not documented; do not send them speculatively to an instrument you rely on.

Function Tokens Beyond the Documented Set

Comparing the keyword dictionary with the documented `[:SENSe]:FUNCtion` list turns up two tokens that are in neither manual. Neither is a usable measurement function — the function parser rejects both — so they are dictionary residue rather than hidden capability:

`TOT2`	A totalize-related dictionary token. It is **not** accepted as a `:FUNCtion` argument (`"TOT2"` returns `-224`); only the documented `TOTalize [1]` selects totalizing.
`TOTOSC`	A totalize-versus-oscillator dictionary token, likewise not accepted as a `:FUNCtion` argument.

Note

A third token found the same way, `VOLTage:PTPeak` (peak-to-peak input voltage), is *not* undocumented and is therefore not covered here: the Programming Guide lists it in the `:FUNCtion` subtree and provides the matching `:MEASure[:SCALar]:VOLTage:PTPeak?` and `:READ` query forms. It is a fully supported function — `:FUNCtion "VOLT:PTP <ch>"` selects it — for which the Programming Guide is the reference.

IEEE 488.2 Macros and the Device Trigger

The firmware implements the full IEEE 488.2 macro set — `*DMC`, `*EMC` / `*EMC?`, `*GMC?`, `*LMC?`, `*RMC`, `*PMC` — together with `*DDT` (Define Device Trigger). These are working on this instrument (verified on a 53131A), but they are **documented IEEE 488.2 common commands, not undocumented features**, and so are only referenced here, not reproduced. The Programming Guide covers them in full: a step-by-step tutorial, “How to Program the Counter to Define Macros,” in Chapter 3; the per-command reference pages in Chapter 4; and the common-command list in Table 2-1.

Two points from that documentation are worth carrying here because they are easy to get wrong on the bench:

- **Macros take parameters.** The body may contain the placeholders `$1` through `$9`, which are substituted from the arguments sent after the label. The Programming Guide's example `*DMC 'setimp',#212:INP1:IMP $1` followed by `setimp 50` sets the Channel 1 input impedance to 50 Ω — confirmed live on a 53131A. An argument is rejected with `-108` only when the body contains *no* placeholder, so a body with a fixed command must be called with no argument.
- `*DDT` **redefines the trigger action.** It stores the command that `*TRG` (and a bus group-execute trigger) performs; the factory value is `#14INIT`, so `*TRG` initiates a measurement, and `*DDT?` reads the action back.

Caution

While macros are enabled (`*EMC 1`), a label that matches a real command header overrides that command. Choose distinct labels so an enabled macro cannot shadow instrument commands still in use, and disable expansion with `*EMC 0` when finished.

The DIAGnostic Service Tree

The documented `:DIAGnostic` subsystem contains only the `:DIAGnostic:CALibration` branch. The firmware's actual command tree contains a large service subsystem under `:DIAGnostic` that no manual mentions, present in every revision. Its twelve branches are listed below with representative leaves. The command paths are as grouped here; the argument syntax is not documented, and these commands drive hardware directly (see the Caution above).

Table 6-1. The Undocumented :DIAGnostic Service Tree

Branch	Representative commands	Function
:SYSTem	:DIAG:SYST:HMACCESS, :NVCOUNT, :PSTARTUP, :DPSOS, :DOUTPUT	Enable hidden menus (Ch.5), NV write count, start-up mode, pSOS debug
:SPI:HDAC	:DIAG:SPI:HDAC:DAC1..DAC6, :HYSTeresis1/2, :INT:OFFSet/RANGe	Direct load of the front-end trigger / hysteresis / interpolator DACs
:SPI	:DIAG:SPI:TDAC1/2, :IACONTROL	Trigger DAC and interpolator-amplifier control
:OPTion	:DIAG:OPT:HFREQuency, :CODE, :HPIB	Channel 3 option / prescaler configuration (Chapter 7)
:HPIB	:DIAG:HPIB:IOCONTROL, :IOSTATUS, :STATus:*, :TEST:*, :DMESsage:*	GPIB interface self-diagnostics and message injection
:MEASure	:DIAG:MEAS:COMPlete, :RESult, :EDETect, :SCOunt, :FAILure:*	Measurement-engine diagnostics
:CALibration	:DIAG:CAL:INTerp:DATA, :ROSCillator:DATA, :TINTerval:DATA, :STATus	Read raw calibration data (partly documented)
:DMESsage / :ISTate / :GPT / :MENU / :DISPlay	:DIAG:DMES:ALL, :ISTate:SCALe, :GPT:COMParator, :MENU:INITialize	Message routing, instrument state, timer comparator, menu init

The most useful leaf is `:DIAGnostic:SYSTem:HMACCESS`, which unlocks the front-panel service menus over the bus (Chapter 5); `:DIAGnostic:OPTion:HFR` sets the Channel 3 option (Chapter 7); and `:ATEST:BTEST` runs the analog board self-test. The `:DIAG:SPI` leaves load the front-end trigger, hysteresis and interpolator DACs directly and must not be sent to a calibrated instrument casually.

Service queries that return data

Several branches answer a query with status or raw data and are safe to read. The responses below were read back from a 53131A (Option 030, no oven oscillator):

Table 6-2. Verified :DIAGnostic Query Responses

Query	Response and meaning
:DIAG:SYST:NVCOUNT?	Two integers — the NV write-count ceiling and the current count (e.g. 30, 27).
:DIAG:SYST:PSTARTUP?	Power-on start-up mode (0 = normal).
:DIAG:SYST:DPSOS? / :DOUTPUT?	pSOS-debug and debug-output flags (0 = off).
:DIAG:OPT:HFR?	Channel 3 option set: prescale, keyword, flag — e.g. 128,N030,1 (Chapter 7).
:DIAG:OPT:CODE?	Numeric option code (2619 = the 32-bit config word at $400016).
:DIAG:CAL:STATus?	Calibration status (1 = valid).
:DIAG:CAL:INTerp:DATA?	Interpolator calibration constants (integer list).
:DIAG:CAL:TINTerval:DATA?	Time-interval calibration constants (integer list).
:DIAG:CAL:ROSCillator:DATA?	Oscillator cal — returns -241 'Hardware missing' with no oven oscillator fitted.
:DIAG:MEASure:RESult?	The last raw measurement result (9.9E37 when over-range / no signal).
:DIAG:MEASure:COMPlete?	Measurement-complete flag (1 = done).
:DIAG:MEASure:SCOunt?	An internal sample / event count.
:DIAG:GPT:COMParator?	Gate / timer comparator state.

Note

Some leaves take an argument: `:DIAG:MEAS:EDETect` and `:DIAG:HPIB:IOSTatus` answer a bare query with `-109` (missing parameter); set-only leaves answer with `-113` (undefined header). `:DIAG:CAL:ROSCillator:DATA?` doubles as a probe for the high-stability oscillator option — `-241` means the oven is not fitted.

Channel 3 input queries

`:INPut3:COUPling?` and `:INPut3:IMPedance?` report the prescaler input's fixed **AC coupling** and **50 Ω** impedance. Both are **query-only** for Channel 3 — a set such as `:INP3:COUP DC` returns `-113` — unlike Channel 1, whose coupling (`DC`) and impedance (1 MΩ) are settable and documented. The array-readout paths also carry undocumented data-type keywords — `BARRay`, `DARRay`, `FARRay`, `IARRay`, `TARRay` (binary / double / float / integer / timed arrays).

The pForth Monitor

The single most powerful undocumented feature — a complete Forth programming environment embedded in the firmware, reachable through the `PFORTH` item of the debug menu — is large enough to warrant its own chapter. Its provenance, how it is reached, its console model and its full word dictionary (including the direct hardware- and EEPROM-access words) are the subject of Chapter 8.

7

The Channel 3 Prescaler Option

How the Option Is Identified

The optional Channel 3 input is a separate plug-in assembly — the A3 Prescaler board — that extends the counter to 3 GHz (Option 030), 5 GHz (Option 050), 12.4 GHz (Option 124) and higher. Each board prescales its input; the counter measures the reduced frequency and scales the reading. The scaling the firmware applies depends on which board is fitted, so the firmware must know which option is present.

Crucially, **the A3 board does not identify itself**. It carries no EEPROM. Two separate facts are handled two different ways:

- **Presence** — whether any Channel 3 board is fitted — is sensed on MC68331 port pins (read at boot from `$FFF907` and `$FFFA19`) and recorded in the flag at `$106990`. This a board can assert with a strap; it says a board is there, not which one.
- **Identity** — which option the board is — is *not* read from the board at all. It is stored in the counter's own non-volatile EEPROM at `$400000` and read back at power-on.

The read is done by the routine at `$02EAC8` (called from boot at `$000D2C`). It first validates the EEPROM — comparing the signature at `$400002` against the string `HP53131 is a winner!` and checking the stored checksum — then copies the option fields out of it:

Table 7-1. Channel 3 Option Fields in the $400000 EEPROM

EEPROM address	Meaning	Cached in RAM
$400000-01	16-bit header checksum (sum of bytes $02-$1F); recomputed on every write	compared at read
$40001A	Status / revision byte	$1038DA
$40001B	Bits 0-6: option code; bit 7: input coupling (0 = DC, 1 = AC)	$1038DF / $1038DE
$40001C	Prescaler ratio / 128 (boot scaling = $1C x 128): 1->128, 4->512	$1038DC
$400016	Stored 32-bit configuration value	compared at read

The low seven bits of the stored byte at `$40001B` are mapped through a small table to an internal option code kept at `$1038DF`. That code is what `*OPT?` turns into the option string it reports (`030`, `050`, `124`, `160`, `200`, or `015`), and it is the index into the scaling table (Table 7-3). If the EEPROM signature fails to validate, the reader substitutes defaults (`$1A=1, $1B=$81, $1C=1`).

The encoding of the option byte is given below. The low seven bits select the option; bit 7 is the input coupling. This is the table the accompanying EEPROM tool uses to read and set the option.

Table 7-2. Channel 3 Option Byte ($1B) Encoding

$1B low 7 bits	*OPT? option	Channel 3 capability
0	015	1.5 GHz (Option 015 - Ch2 on 53181A)
1	030	3.0 GHz (Option 030)
2	050	5.0 GHz (Option 050)
3	124	12.4 GHz (Option 124)
4	160	16.0 GHz (Option 160)
5	200	20.0 GHz (Option 200)
6 or >6	030	defaults to 3.0 GHz

Bit 7 of `$40001B` set means AC input coupling, clear means DC. The 32-bit value at `$400016` and the byte at `$40001C` are board-specific and are preserved by the tool when only the option is changed.

Caution

Because identity is stored in the counter — not on the board — swapping A3 boards is **not** plug-and-play. Fitting a different prescaler does not change what the counter believes is installed; the stored option must be updated to match (see below). Equally, a full non-volatile-memory clear (Chapter 3) erases the stored option along with everything else, after which the Channel 3 option must be re-entered.

A blank header on units shipped without the option

A counter that never shipped with the optional channel may carry a **completely blank header**: the 32-byte record at `$400000` is all `$FF` — no signature, no checksum and no option bytes — even though the calibration records from `$400020` onward are present and valid. The header was simply never written at the factory because there was no option to record. At power-on the reader at `$02EAC8` finds the signature absent, treats the store as carrying no option, and substitutes the defaults noted above (`$1A=1, $1B=$81, $1C=1`), so `*OPT?` reports nothing on Channel 3 whatever board is fitted.

Enabling the option on such a unit therefore means writing the **whole header**, not just the option byte: the signature `HP53131 is a winner!` at `$02`, the 32-bit configuration word at `$16`, the status byte `$1A`, the option byte `$1B` and prescale `$1C` (Tables 7-1, 7-2), and the header checksum at `$00`. The configuration word is a fixed record marker — `$00000A3B` (2619, the value `:DIAG:OPT:CODE?` returns) — identical across every factory option dump examined, not an option-specific value.

Note

Both routes for setting the option handle this automatically; the blank header needs no separate repair step. Over GPIB, `:DIAGnostic:OPTion:HFR` builds and commits the full 32-byte header: the firmware's header builder copies the signature from ROM on every write and fills the default status and prescale for a previously blank header, so setting the option (next section) writes a valid header as a side effect. Offline, the companion `531xx_eeprom_tool.py` detects the missing signature when the option is applied and reconstructs the same header — signature, configuration word, status and checksum — around the chosen option, leaving the calibration records untouched. Because those records survive, a unit repaired this way keeps its existing calibration; only the option identity was missing.

The Per-Option Scaling Table

The constant the firmware applies for each option is not on the board; it is hard-coded in the main ROM, in a six-entry table of 8-byte IEEE-754 doubles at `$0270D0`, indexed by the *internal* option code (the value the reader derives from `$1B`, not the `*OPT?` number), so its row order does not line up one-for-one with Table 7-2. Its six entries are:

Table 7-3. Channel 3 Per-Option Constant ($0270D0)

Internal code	Value (double)	*OPT? it maps to
0	10 000 000	030
1	10 000 000	015
2	500 000 000	030
3	1 x 10^-7	050
4	1 x 10^-7	124
5	2 x 10^-9	160

These are per-option reference constants, loaded during measurement setup alongside the per-option frequency limits — a separate table at `$02D680` (Table 7-6). The two magnitudes — a reference frequency for the direct-prescale options (10 MHz, 500 MHz), its reciprocal for the higher bands (1 x 10^-7, 2 x 10^-9) — reflect the two ways the reading is formed. They do **not** scale the displayed frequency: that is the prescale. Options 015 and 030 hold different constants here yet read a given input identically at the same prescale, so the constant is a measurement-setup reference, not the frequency factor. For setting the option, Table 7-2 (the `$1B` encoding) is the one to use.

Caution

This table is in ROM. It cannot be changed without reprogramming the EPROMs, and it should not need to be: it already carries the correct constant for every option the firmware supports. Making a board read correctly is a matter of setting the board's stored identity to one of these options, not of altering this table.

Setting the Option over GPIB

The firmware contains an **undocumented service command that writes the stored Channel 3 option** into the counter's $400000 EEPROM — the mechanism by which the counter is told which prescaler is installed. It lives in the undocumented `:DIAGnostic:OPTion` subtree (Chapter 6), whose three members are:

`:DIAG:OPT:HFR (HFREQuency)`	Writes the option/configuration set into the counter's non-volatile EEPROM. Its handler (at `$027BAA`) builds a configuration record and commits 32 bytes to the EEPROM at `$400000` through the RAM-resident byte-write routine; a failed write raises the internal message `EEPROM FAIL`.
`:DIAG:OPT:CODE`	A sibling node addressing the option code directly.
`:DIAG:OPT:HPIB`	A sibling node in the same service group.

The short form is `HFR` (not `HFREQ`), and it takes **three** arguments in this order:

```
:DIAGnostic:OPTion:HFR <prescale>, <option>, <coupling>
```

`<prescale>` — The **Channel 3 prescaler divide ratio** — the number the counter multiplies the prescaled reading by to recover the true frequency. For the 3 GHz Option 030 board this is **128** (a 100 MHz input divides to 781.25 kHz at the counter; 781.25 kHz x 128 = 100 MHz). The counter stores it as the single byte `$1C` = prescale / 128 (so 128 stores as 1, 512 as 4) and, at boot, recovers the ratio as `$1C` x 128. Range 128-16384. This is the value that must match the fitted board.

`<option>` — The option keyword. The firmware spells the option numbers with a leading N so they are valid mnemonics: `N015`, `N030`, `N050`, `N124`, `N160`, `N200` or `NONE`. This sets the option code in `$1B` and becomes the `*OPT?` Channel 3 field.

`<coupling>` — A stored flag written to bit 7 of `$1B` that the boot code reads as an input-coupling default. `:DIAG:OPT:HFR?` echoes the value written (0 or 1), but on a prescaler board it has no visible effect: the Channel 3 input is AC-coupled and `:INPut3:COUPling?` reports `AC` either way (Chapter 6). Its factory value is not uniform — the 3 GHz (030) units examined store `1`, the 1.5 GHz (015) boards store `0` — and because the flag is inert on a prescaler channel, either value works; use `1` to match a 3 GHz unit.

The full command to configure the 3 GHz board is `:DIAG:OPT:HFR 128,N030,1`, which is what `:DIAG:OPT:HFR?` returns for a factory 3 GHz unit. The prescale scales the reading directly: with a 5 GHz configuration (`512,N050,1`) stored on a 3 GHz board, a 120 MHz Channel 3 input reads **480 MHz** (120 MHz / 128 x 512); the correct `128,N030,1` reads it as 120 MHz.

All three arguments are mandatory. The parser accepts the full triple and nothing shorter — there is no short form and no default fill. The prescale must come first as a number, the option must be a keyword, and the coupling flag must be present.

Table 7-4. HFR Argument Rules

Argument	Required	Accepts	Rejects (error)
<prescale>	yes	integer 128-16384	<128 or >16384 (-222); missing (-109)
<option>	yes	N015 / N030 / N050 / N124 / N160 / N200	a number (-128); unknown keyword (-224)
<coupling>	yes	stored flag, 0 or 1 (bit 7 of $1B)	missing (-109)

Fewer than three arguments returns `-109,"Missing parameter"` and writes nothing; a fourth returns `-108,"Parameter not allowed"`; leading with the option keyword returns `-148,"Character data not allowed"`. In every rejected case `:DIAG:OPT:HFR?` is unchanged, so a malformed command is safe — it is refused, not half-applied. The prescale need not be a power of two, but every real board divides by one, so only powers of two are meaningful.

Caution

This command rewrites the counter's calibration/configuration EEPROM. A wrong option selector will make the counter apply the wrong scaling to every Channel 3 reading, and the error persists across power cycles because it is stored non-volatile. Record the present `*OPT?` string before writing, change one thing at a time, and re-read `*OPT?` and a known-frequency reading afterwards to confirm.

Note

Match the prescale and the option keyword to the *same* board. The option keyword sets the frequency ceiling (Table 7-6) as well as the `*OPT?` string, while the prescale sets only the divide ratio, so the two arguments must describe one real board. A field case makes the trap concrete: a 12.4 GHz board fitted where the firmware still held option 030 was given the correct ÷512 prescale but kept the 030 keyword — `:DIAG:OPT:HFR 512,N030,1`. Option 030's 3.5 GHz ceiling then rejected the 10 GHz input as over-range (`9.9E37`), so Channel 3 produced no valid reading at all: `:SYST:ERR?` returned `-230,"Data corrupt or stale"` and the display showed only dashes, unchanged by a power cycle because the mismatched pair is stored non-volatile. Writing the matching pair — `:DIAG:OPT:HFR 512,N124,1` — restored a correct reading (10.123 GHz in, 10.123 GHz displayed). Set both fields together to the fitted board, and confirm with a known Channel 3 frequency afterwards. The companion `531xx_gpib_tool.py` now warns when a measured ratio does not match the option keyword about to be stored, and steers the choice to the option that matches.

The Prescaler Ratio for Each Option

The prescaler divide ratio is stored in the counter as the single EEPROM byte `$40001C` = ratio / 128, and the boot code recovers the ratio as `$1C` x 128 (see Table 7-1). The ratio for each option is:

Table 7-5. HFR Arguments and Prescaler Ratio per Channel 3 Option

*OPT?	<option>	Board	<prescale>	$1C byte
015	N015	1.5 GHz	128	$01
030	N030	3 GHz	128	$01
050	N050	5 GHz	512	$04
124	N124	12.4 GHz	512	$04
160	N160	16 GHz (53181A)	512+	$04+
200	N200	20 GHz (53181A)	512+	$04+

The ratios follow the shared-hardware groupings. The 1.5 GHz and 3.0 GHz boards use one prescaler assembly and divide by 128; the 5 GHz and 12.4 GHz boards use another and divide by 512. Each pair is a single assembly with a different input band — the lower-frequency option is the higher board with an input limiter — so the two options of a pair share a prescaler ratio and differ only in the frequency ceiling below. A board's ratio can always be read directly from its `$1C` byte, or measured (below).

Note

Options 160 (16 GHz) and 200 (20 GHz) were never offered as products — the manufacturer's Channel 3 lineup ends at 12.4 GHz (124). Both are firmware provisions only: a keyword, option code, `*OPT?` string and frequency ceiling exist for each, but no board was made, so no prescale ratio is defined for them. Option 200 is further incomplete — it has no entry in the `$0270D0` scaling table (Table 7-3 ends at 160), so a 200-configured unit cannot form a valid Channel 3 reading. They are retained in these tables only to document the firmware, not because usable hardware exists.

Measuring a board's divide ratio

Where a board cannot be dumped, its ratio can still be read off the bench in a single pass, because the firmware multiplies the recovered reading by the stored prescale:

1. Enable service access: `:DIAG:SYST:HMACCESS 1`.
2. Store a trial ratio P — start at 128: `:DIAG:OPT:HFR 128,Nxxx,1` (use the option keyword for the fitted band; the third argument is the stored coupling flag — 1 matches the factory setting).
3. Apply a known frequency F to Channel 3 from a trusted source, within the board's range.
4. Read Channel 3: `:FUNC "FREQ 3"` then `:READ?` — call the result R.
5. The board's true divide ratio is **N = P x F / R**, rounded to the nearest power of two (prescalers divide by a power of two, and the firmware accepts 128-16384).
6. Store it: `:DIAG:OPT:HFR N,Nxxx,1`, then re-read Channel 3 — it should now report F exactly.

Note

Worked example: with P = 128 and a 120 MHz input, a correctly-scaled board reads R = 120 MHz, giving N = 128 x 120 / 120 = 128. A board that divides by 256 with 128 stored would read 60 MHz, giving N = 128 x 120 / 60 = 256 — the reading is a power-of-two multiple of the input whenever the stored ratio is wrong.

The Per-Option Frequency Ceiling

The prescaler ratio is not the only thing the option code controls. The firmware also holds a **per-option maximum input frequency**, and when a Channel 3 reading exceeds it the counter returns the IEEE over-range value **9.9E37** in place of the true frequency. Because the 1.5 GHz and 3.0 GHz boards are physically identical (same prescaler assembly), this ceiling — not the hardware — is what actually separates the two options.

The limit table is in ROM at `$02D680`, eight-byte doubles indexed by the internal option code. The check at `$043A9C` loads the option's limit, compares the measured Channel 3 frequency against it (double compare at `$05AD94`) and substitutes `9.9E37` when the input is higher. Each ceiling is the option's nominal top frequency plus roughly ten percent:

Table 7-6. Per-Option Frequency Ceiling ($02D680)

*OPT?	Nominal	Firmware ceiling	Above the ceiling
015	1.5 GHz	1.7 GHz	Channel 3 reads 9.9E37
030	3.0 GHz	3.5 GHz	Channel 3 reads 9.9E37
050	5.0 GHz	5.5 GHz	Channel 3 reads 9.9E37
124	12.4 GHz	13.4 GHz	Channel 3 reads 9.9E37
160	16 GHz	16.5 GHz	Channel 3 reads 9.9E37
200	20 GHz	22 GHz	Channel 3 reads 9.9E37

The useful part is the consequence. Because the 1.5/3.0 GHz hardware is identical and only the firmware ceiling differs, writing option 030 to a 1.5 GHz unit — `:DIAG:OPT:HFR 128,N030,1` — raises the ceiling from 1.7 GHz to 3.5 GHz and turns it into a working 3 GHz counter with no hardware change. The confirmed 5 GHz / 12.4 GHz pair is the same story one band up: identical ÷512 hardware, different firmware ceiling (5.5 vs 13.4 GHz), so option 124 on a 5 GHz unit would likewise lift the cap.

Note

The ceiling is indexed by the *internal* option code, so the two encodings of 030 differ: the normal one (`$1B` code 1, written by `N030`) caps at 3.5 GHz, while the fallback encoding `$1B` = `$06` — which also reports `*OPT? 030` but is reachable only by writing the byte directly with the EEPROM tool — selects the default limit entry of 2 x 10^11 Hz, effectively no ceiling. On a 3 GHz board the prescaler's own bandwidth is then the real limit.

Note

A 3 GHz (030) board configured as 015 therefore reads Channel 3 normally up to ~1.7 GHz and returns `9.9E37` above it; configured as 030 it reads to the board's own bandwidth (the 3.5 GHz firmware ceiling is above what a 3 GHz board can physically reach). Two companion scripts exercise the ceiling: `531xx_ceiling_prescale_test.py` confirms the limit values with only a low-frequency source — raising the stored prescale until the reconstructed reading crosses each limit — and `531xx_ch3_ceiling_test.py` sweeps a real signal from an HP/Agilent 8648C generator.

Practical Guidance

To match the firmware's scaling to a particular Channel 3 board:

1. Power on with the board fitted and query *OPT?. If it already reports the option that matches the fitted board (030 / 050 / 124 / ...), the stored setting is correct and no action is needed.
2. If *OPT? is blank or wrong, the option stored in the counter's EEPROM does not match the fitted board. Determine the board's true option from its A3 assembly part number (it encodes the option) and the service-manual option list.
3. Write the matching option and prescale into the counter with `:DIAGnostic:OPTion:HFR <prescale>,<Nxxx>,1` (Table 7-5), then power-cycle so the boot code re-reads the stored value.
4. Re-query *OPT?, apply a reference signal of known frequency to Channel 3, and confirm the displayed frequency is correct before trusting the unit.

Note

The same EEPROM can also be read and written from the pForth monitor (Chapter 8), whose `wr_eeprom` word reaches the `$400000` store directly. That route is lower-level and carries no range checking, and a careless write there corrupts calibration as readily as configuration; the `:DIAG:OPT:HFR` command is the safer of the two because it validates its arguments and recomputes the checksum.

Note

Two companion tools accompany this document. `531xx_gpib_tool.py` does this live over GPIB or RS-232: it scans the bus, identifies the counter, and issues `:DIAGnostic:OPTion:HFR` (with the prescale and coupling of Table 7-5) plus the verification described above. `531xx_eeprom_tool.py` does it offline on a saved EEPROM dump: it decodes `$1B` and `$1C` per Tables 7-1/7-2, writes the new option and prescale and recomputes the header checksum for programming back.

8

The Embedded pForth Monitor

What It Is

The firmware embeds **pForth** — the public-domain Forth by Phil Burk — as a complete, interactive programming environment running as a task under the instrument's pSOS real-time kernel. It is a full Forth: a dictionary of words, a text interpreter and a compiler, together with a large set of words that reach the counter's hardware and non-volatile memory directly. The image carries the source-control banner `pForth $Revision: 1.1 $`.

It is present across the family. Every image examined contains it: the 53132A (revisions 3703, 3944, 4613) and the 53181A (3703, 3944, 4243) carry the `$Revision: 1.1 $` build; the earliest 53131A firmware (3427) carries `$Revision: 1.3 $`. The dictionary and addresses in this chapter are from the 53132A revision 4613 reference image; the word set is materially the same in the others.

Note

`$Revision:` is an RCS/CVS source-control keyword — the revision of the pForth source file compiled in, not a pForth release number. The `1.3` in the 3427 image is simply a different file revision, not a newer interpreter than `1.1`.

Reaching the Monitor

The monitor is enabled from the **hidden debug menu** (hold `+/-` at power-on; Chapter 5). Because the debug menu is gated, the hidden-menu-access flag must be set first with the remote command `:DIAGnostic:SYSTem:HMACCESS 1` (Chapter 5); until then the `+/-` key does nothing at power-on. In the menu the `+/-` key steps between items and the arrow keys change the current item; step to the `PFORTH:` item and use an arrow key to toggle it `ON`. `PFORTH:` is a stored on/off setting, not a one-shot action.

Once enabled, the interpreter runs as an **additional pSOS task** — named `p4th` — alongside normal operation. The counter keeps measuring and keeps answering GPIB; the Forth prompt appears on the serial port described next, and stays available across power cycles until `PFORTH:` is toggled off again. This is why enabling it has no visible effect on the front panel or the bus.

Caution

The monitor has **no guard rails**. Its words write hardware DACs, the display, the FPGA and the `$400000` EEPROM with no range checking and no confirmation. A mistaken `wr_eeprom` or `hdac_write` can corrupt calibration beyond what the menus can undo. Explore with the read and print words; record original contents before writing anything.

The Console and I/O Model

pForth reads a line of input with `expect`, breaks it into tokens with `word`, and prints with `emit`, `type` and `."`. Input and output pass through a pSOS device layer (`dev_open`, `dev_read`, `dev_write`, `dev_close`, `dev_ctrl`) selected by the word `!iodev`; the compiled-in start-up runs `0 !iodev`. **Device 0 is the serial port** — the MC68331's on-chip SCI, brought out to the rear-panel RS-232 connector. The interpreter is driven from a terminal on that port; GPIB is left free to serve normal SCPI.

This is confirmed on a running unit. In the kernel's own process listing the `p4th` task holds the SCI-read resource (`sciR`) and moves characters through the `sciR` and `sciW` message exchanges; a failed transfer surfaces as `I/O error / Device: / Function:`. So although the dictionary also carries GPIB words (`hpib_stat`, `hpib_ioctrl`, `dmes_hpib`), the interactive console itself is the serial line.

Connecting a terminal

Connect at **9600 baud, 8 data bits, no parity, 1 stop bit** — the counter's RS-232 default (set in the Utility menu, Chapter 5). The counter's serial port is wired as **DTE** (it is meant to drive a printer), so a null-modem / crossover cable is required between it and a PC. With a terminal attached and `PFORTH:` enabled, pressing Return brings up the prompt `p4th D >` — `p4th` is the task name and `D` the current number base (`D` decimal, `X` hex). The `ver` word reports the build:

```
p4th D > ver

pForth $Revision: 1.3 $

pSOS version ID: 3341

p4th D > 1 2 + .

 3

p4th D >
```

Note

This transcript is from a 53131A, whose compiled-in pForth source is revision 1.3; the 53132A revision 4613 reference image used elsewhere in this chapter reports `$Revision: 1.1 $` instead. The dictionary and behaviour are the same.

The image also contains live Forth source, compiled in, that shows the style of the environment — it defines two words that prompt on the console and drive the display (both appear at the top of a `words` listing):

```
variable dstring 30 allot

: newdisp

  ." Enter string (CAPS): "  dstring 30 expect  dstring disp_string ;

: setstr

  ." Enter value for dstring (CAPS): "  dstring 30 expect ;

0 !iodev
```

`newdisp` prompts for text, reads up to 30 characters with `expect`, and writes them to the vacuum-fluorescent display with `disp_string`. The `(CAPS)` in the prompt is a reminder that entry is upper-case.

The pSOS task set

The monitor's own reporting words expose the whole firmware architecture. `ps` lists the pSOS tasks and message exchanges, `mem_rep` the memory regions. On a running 53131A the task set is:

Table 8-1. pSOS Tasks (from the live ps listing)

Task	Priority	Role
p4th	250	The pForth interpreter itself; holds the serial console (sciR)
hpib	249	GPIB / IEEE-488 interface and its message buffers
frpn	150	Front panel (keyboard and display)
setu	100	Measurement setup
mast	50	Master measurement task
slav	25	Slave measurement task
ROOT	1	pSOS root task
IDLE	0	Idle task

The tasks communicate through pSOS message exchanges — `meas`, `setu`, `sciW` and `sciR` — the last two being the serial console's write and read queues. The `dmes_frpn`, `dmes_mast`, `dmes_slav` and `dmes_setu` hardware words (Table 8-4) each take a one-word message code and send it to the correspondingly-named task; `iac_read` is a two-argument read-only register read.

The Dictionary

The vocabulary falls into three parts: a standard Forth core, the pSOS kernel interface, and the counter-specific hardware words. Every word below is taken verbatim from the dictionary in the reference image.

Standard Forth core

Table 8-2. pForth Core Words

Group	Words
Stack	`dup ?dup drop 2drop swap 2swap over 2dup rot -rot pick depth >r r>`
Arithmetic / logic	`+ - * / mod /mod */mod */ 1+ 1- 2+ 2- 2* 2/ negate abs max min and or xor not = < 0< 0= 0> << >> sin cos`
Memory	`@ ! c@ c! w@ w! +! here allot fill cmove cmove> pad tib count`
Defining / dictionary	`variable constant create literal : ; , w, c, ' ['] [ ] does> recurse immediate user fence unfence forget words`
Control flow	`if else then endif begin while repeat again until do ?do loop +loop leave i j k`
Numeric & text I/O	`decimal hex base number 0x . u. u.x .s emit space spaces cr ." (.") type word expect key`
Interpreter / system	`interpret execute startup halt abort false true state ver watch root ps mem_rep s_rep`

Numbers default to decimal; `hex` and `decimal` switch the base and `0x` reads a hex literal inline. `.s` prints the stack non-destructively and `words` lists the dictionary — both safe first commands to confirm the monitor is live. Trig is integer fixed-point: `sin` and `cos` take an angle in whole degrees and return the value scaled by 10000 (`90 sin` gives `10000`, `45 sin` gives `7071`). The three reporting words dump the kernel state — `ps` the task table and message exchanges, `mem_rep` the memory regions, and `s_rep` each task's user and supervisor stack usage.

pSOS kernel interface

These words are a thin Forth veneer over the pSOS real-time kernel the firmware runs on, and are what make the environment a genuine systems-debug tool rather than a toy calculator. They manage tasks, semaphores, message exchanges and device drivers.

Table 8-3. pSOS Kernel Words

Group	Words
Tasks	`my_pid spawn delete suspend resume priority pause mode ident ?^C jam_x create_x delete_x`
Messages / semaphores	`send_x request_x signal_v wait_v get_v`
Device drivers	`dev_init dev_open dev_close dev_read dev_write dev_ctrl !iodev`

Counter hardware words

This is the group with no analogue anywhere in the standard manuals: named words that drive the counter's front end, converters, display, interface and non-volatile store directly.

Table 8-4. Counter Hardware Words

Subsystem	Words
Front end (signal conditioning)	`fe_set_coupling_dc fe_set_impedance_50ohm fe_set_attenuator_x1 fe_set_filter fe_set_sensitivity fe_set_com fe_set_trigger_rise fe_set_trigger_abs`
Front-end cal & peak measure	`fe_cal_offset fe_cal_gain fe_measure_low_peak fe_measure_high_peak fe_peak_between fe_tdac_ramp`
DACs / FPGA / timebase	`hdac_write hdac_all tdac_write tdac_all cmp_fpga c_prog_fpga fpga_wr16 tbsel pllinh int_tb_ctrl`
Interpolator / analog	`iac_read iac_write otc_write`
Display / front panel	`dmessage disp_string set_blink update_digit fp_blank vfd_write led_write led_all cmp_ist_dup`
GPIB interface	`hpib_stat hpib_rem hpib_iostat hpib_ioctrl dmes_hpib dmes_frpn dmes_slav dmes_mast dmes_setu dmes_all`
EEPROM / non-volatile	`wr_eeprom`
Diagnostic print	`pr_max_nv pr_max_sav_rcl pr_eeprom_debug debug_psos debug_fr_end infr`

The most useful — and most dangerous — words

`fe_set_coupling_dc`
`fe_set_impedance_50ohm`
`fe_set_attenuator_x1`
`fe_set_sensitivity`
`fe_set_filter`
	Drive the Channel 1/2 front-end signal-conditioning relays and DACs directly. `fe_set_coupling_dc` takes two arguments, `<channel> <enable>` (verified live: `1 1 fe_set_coupling_dc` audibly throws the Channel-1 coupling relay to DC). These words act **below the SCPI and front-panel layer**: the relay changes and the reading follows, but `:INPut1:COUPling?` still reports the configured value and the AC/DC annunciator does not move. The effect is also transient — while a measurement is running the setup task re-applies the configured front end each cycle, so to hold a change, stop measuring first (`:INITiate:CONTinuous OFF`).

`fe_measure_low_peak` `fe_measure_high_peak`	Read the input peak detectors — the hardware behind the `VOLT PEAKS` function. Called as `<channel> fe_measure_high_peak`, they return the peak voltage in **millivolts** (verified live: a 1 V peak reads `989`, 0.5 V reads `489`). Read-only and safe.
`fe_cal_offset` `fe_cal_gain`	Run the front-end offset and gain calibration steps, one channel at a time (`<channel>`). Verified live: `1 fe_cal_offset` re-ran the Channel-1 offset cal and moved block word 0 (the Ch1 offset trim, `2048` to `2046`); `1 fe_cal_gain` needs a reference signal applied and makes no change without one. Both write the calibration constants **below the security layer** — they run while the unit is secured — and **without bumping the calibration count**, so a unit adjusted this way carries no record of it. Save `:CALibration:DATA?` first (Chapter 3).

`disp_string`
`update_digit`
`led_write`
`set_blink`
`fp_blank`
	Write the vacuum-fluorescent display, its digits and the annunciator LEDs directly. `disp_string` takes the address of a string (as `newdisp` uses it); `update_digit` takes `<position> <value>`; `led_write` takes a one-word annunciator bitmask (verified live: `255 led_write` lights the annunciators); `set_blink` takes two arguments and `fp_blank` one. Harmless, and the easiest way to see the monitor executing your words — a refresh from the measurement task restores the normal display.

`pr_max_nv`
`pr_max_sav_rcl`
`pr_eeprom_debug`
	Print the non-volatile write count, the maximum save/recall register count, and a formatted dump of the EEPROM debug fields. Read-only — safe.

`wr_eeprom`	Writes the `$400000` non-volatile store directly, with no range check and no checksum fix-up: change a header byte and the checksum no longer matches, so the firmware discards the store and rebuilds defaults at the next power-up. This is the word that can reach the calibration constants and security code outside the menus, and the one most able to brick a unit's calibration. The same store — and any hardware register — is also reachable with the standard Forth fetch/store words; see below.

The whole `fe_set_*` family shares the `<channel> <value>` form and was exercised on a running 53131A against a known Channel-1 signal. Each throws a real relay or DAC and the peak detector confirms the change, while the SCPI model and the front-panel annunciators stay put:

Table 8-5. Front-End Control Words (verified on hardware)

Word	Args	Verified effect
fe_set_coupling_dc	ch, enable	Coupling relay; enable=1 → DC (audible click; the reading then follows a DC offset).
fe_set_impedance_50ohm	ch, enable	Input impedance; enable=1 → 50 Ω. The peak detector sits on the 1 MΩ path, so it reads 0 while 50 Ω is engaged.
fe_set_attenuator_x1	ch, enable	The ÷10 (20 dB) input attenuator; enable=1 → ×1, enable=0 → ÷10 (a 0.5 V peak read 489, then 39).
fe_set_filter	ch, enable	The 100 kHz low-pass; enable=1 inserts it (a 1 MHz input read 500, then 19). The peak detector is after the filter.
fe_set_sensitivity	ch, level	Trigger sensitivity / hysteresis; the second argument selects the level.

Caution

Group the words by consequence. Reading and printing (`words`, `.s`, `pr_max_nv`, `pr_eeprom_debug`, the `fe_measure_*` words) changes nothing and is the right way to learn the monitor. Writing (`wr_eeprom`, `hdac_write`, `tdac_write`, `fpga_wr16`, the `fe_set_*` words) changes hardware or non-volatile state immediately. Before any write, capture the current calibration with `:CALibration:DATA?` (Chapter 3) so the unit can be restored.

A worked session

The following is a real session on a 53131A with a known signal on Channel 1, demonstrating a colon definition, the peak-detector read, and a live front-end relay change (the input was a 1 Vpp sine — 0.5 V peak — with a +0.5 V offset):

```
p4th D > : sq dup * ;              ( define a squaring word )

p4th D > 7 sq .

 49

p4th D > 1 fe_measure_high_peak .  ( Ch1 high peak, mV, AC-coupled )

 489

p4th D > 1 1 fe_set_coupling_dc    ( throw the coupling relay to DC )

p4th D > 1 fe_measure_high_peak .  ( now follows the DC offset )

 979

p4th D > 1 0 fe_set_coupling_dc    ( relay back to AC )
```

The peak reading tracks the coupling relay while the SCPI model and the AC/DC annunciator do not — the clearest illustration of how far beneath the instrument's normal control surface these words reach.

Reading and writing memory directly

Because pForth runs on the bare processor, its ordinary fetch and store words — `@`, `!`, `c@`, `c!` (and `w@` / `w!`) — address the MC68331's entire memory-mapped space, hardware included. The `$400000` EEPROM is simply memory, and reads back byte-for-byte:

```
p4th D > hex 400000 c@ .  400001 c@ .   ( the header checksum )

 9 2B

p4th D > : sig 14 0 do 400002 i + c@ emit loop ;  sig

HP53131 is a winner!
```

`c!` writes it just as directly. On a running unit `AA 40001D c!` set an unused header byte, `40001D c@` read the `AA` back, and `FF 40001D c!` restored it. This is the most powerful — and most dangerous — capability in the instrument: calibration, configuration and live hardware registers are all one `c!` away, with nothing between the keyboard and the bus, no security check and no checksum fix-up. Keep a `:CALibration:DATA?` backup (Chapter 3) before writing anything, and put an altered header byte back before power-cycling so the header checksum stays valid — otherwise the firmware discards the store and rebuilds defaults at the next power-up.